

Vista general de los lenguajes de programación

Historia, abstracción y paradigmas

Reporte 2

Angel Rugerio Jiménez — 201720

Otoño 2026



```
total = sum(xs)
```

Muy alto nivel

```
x = a + b;
```

Alto nivel

```
MOV AL, 61h
```

Ensamblador

```
10110000 01100001
```

Código máquina

↑ más abstracción

De qué vamos a hablar

01

Historia

Pioneros, hitos, línea del tiempo y tendencias

02

Proceso de abstracción

Qué es abstraer y sus niveles

03

Abstracciones comunes

Variables, tipos, funciones, objetos, módulos

04

Tipos de lenguajes

Bajo nivel, alto nivel y DSL

05

Paradigmas

Imperativo, objetos, funcional, lógico, multiparadigma

06

Conclusiones

Qué nos llevamos y referencias

¿Qué es un lenguaje de programación?

“

Una notación formal para describir cálculos de forma que puedan ser leídos por personas y ejecutados por una máquina.

Louden y Lambert (2011)

Doble audiencia → cada generación acerca la notación al problema y la aleja del hardware. Ese movimiento es la abstracción.

Relación con la materia

Tipo 2

Gramática libre de contexto

Define la sintaxis. ALGOL 60 fue el primero en especificarse así, con BNF.

Tipo 3

Expresiones regulares y autómatas finitos

Construyen el analizador léxico (Aho et al., 2006).

Los lenguajes pioneros (1940–1960)

~1949

Máquina y ensamblador

Binario →
mnemónicos (MOV,
ADD) y etiquetas.
Atado a una
arquitectura.

Sebesta (2019)

1957

Fortran

Backus, IBM. Primer
compilador de alto
nivel casi tan
eficiente como el
código a mano.

Backus et al. (1957)

1958–60

Lisp

McCarthy. Listas,
recursión, funciones
como valores,
recolector de basura.

McCarthy (1960)

1960

ALGOL 60

Sintaxis formal con
BNF. Bloques
anidados, alcance
léxico. Ancestro de
Pascal y C.

*MSMK (2025); Sebesta
(2019)*

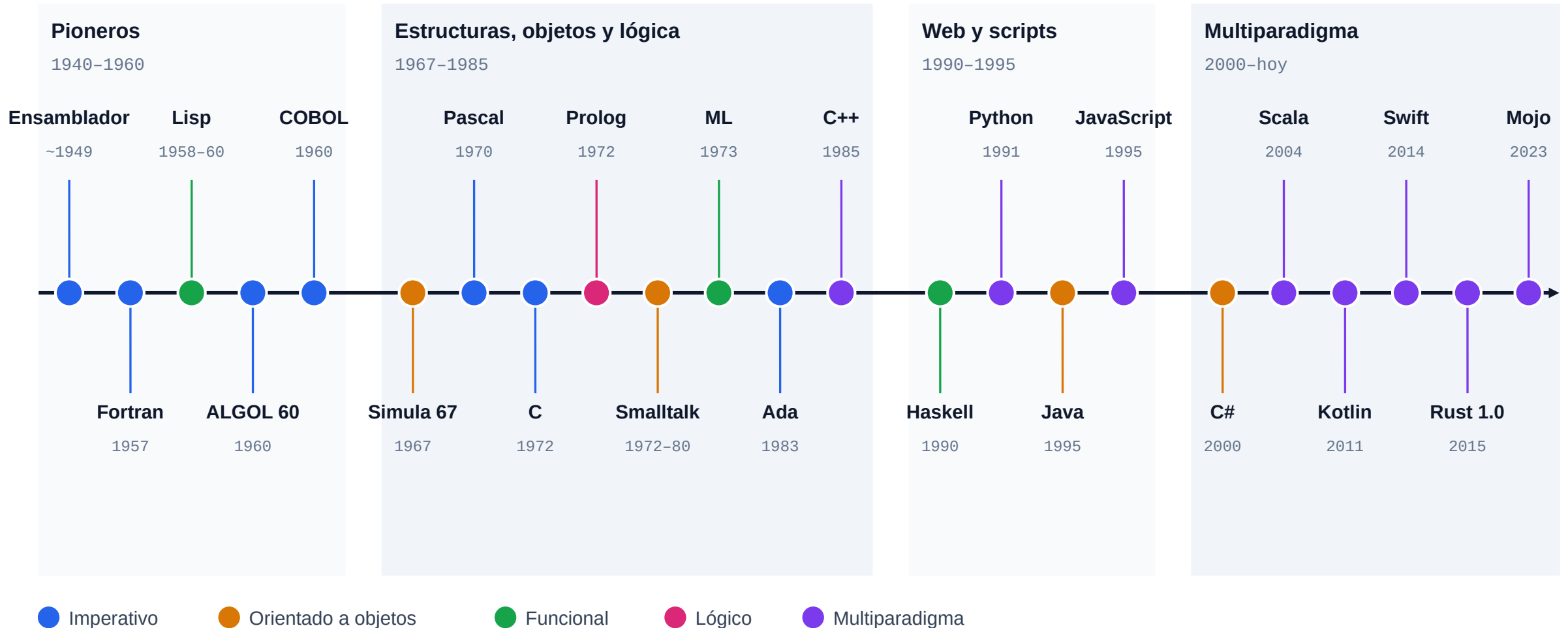
1960

COBOL

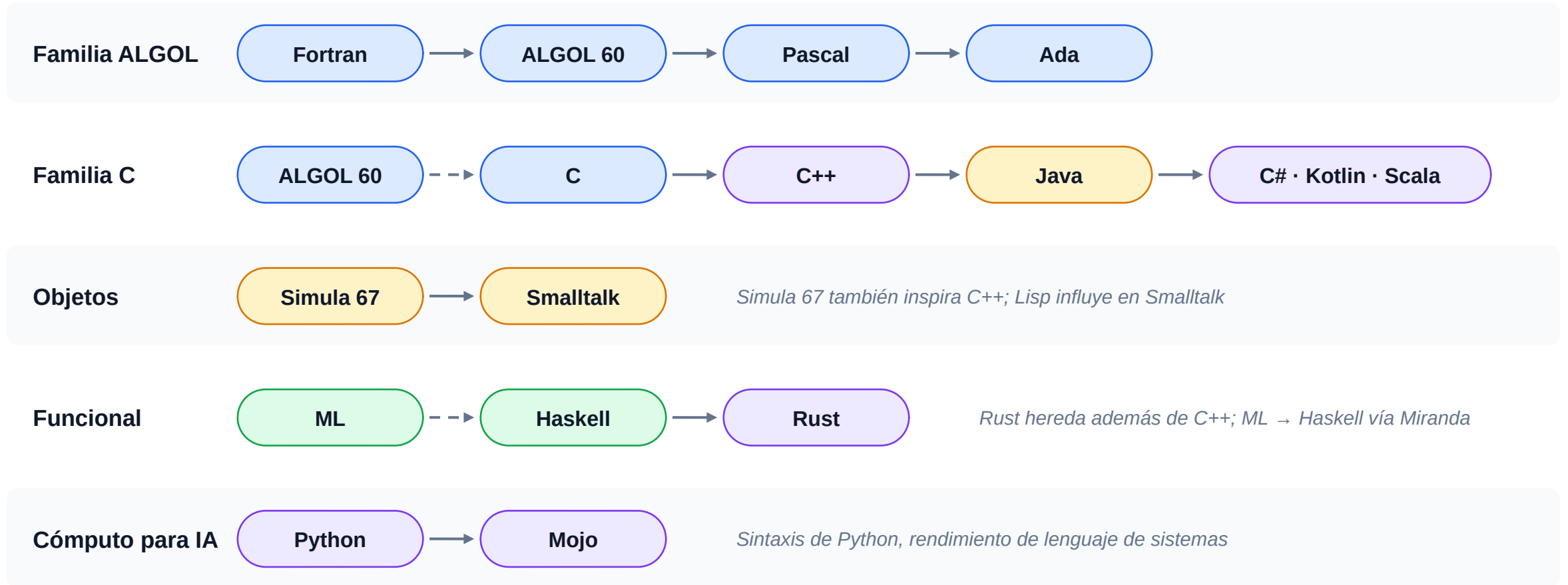
CODASYL. Datos de
negocios, sintaxis
cercana al inglés,
registros.

Sebesta (2019)

Línea del tiempo: 75 años de lenguajes



Linajes: quién influyó en quién



Línea punteada = influencia indirecta (ALGOL 60 → CPL → BCPL → B → C; ML → Miranda → Haskell).

Hitos: cada etapa sumó una abstracción



Tendencias actuales

1

Multiparadigma como norma

Scala, Kotlin, Swift, Rust; Java 8 y C++11 incorporan lambdas.

Van Roy (2009)

2

Seguridad de memoria sin recolector

El ownership de Rust elimina errores de memoria desde la compilación.

Rust 1.0 · 2015

3

Lenguajes para IA

Python domina por su ecosistema; Mojo busca su productividad con rendimiento de sistemas, p. ej. en finanzas.

Han (2026)

4

DSL también para IA

NineToothed: núcleos de GPU con semántica secuencial sobre Triton. Anka: sintaxis restringida para que los LLM generen código fiable.

Huang et al. (2025); Al Mazrouei (2025)

¿Qué significa abstraer?

“Un proceso mental que consiste en realzar los detalles relevantes [...] mientras se ignoran los detalles irrelevantes.”

Garrido (s.f., p. 1), Universidad de Granada · ver también Louden y Lambert (2011)

Abstracción de procesos

Encapsular una secuencia de acciones bajo un nombre. Se sabe qué hace, no cómo.

Subprogramas · funciones · métodos

La primera que aparece en la historia (Garrido, s.f., p. 3)

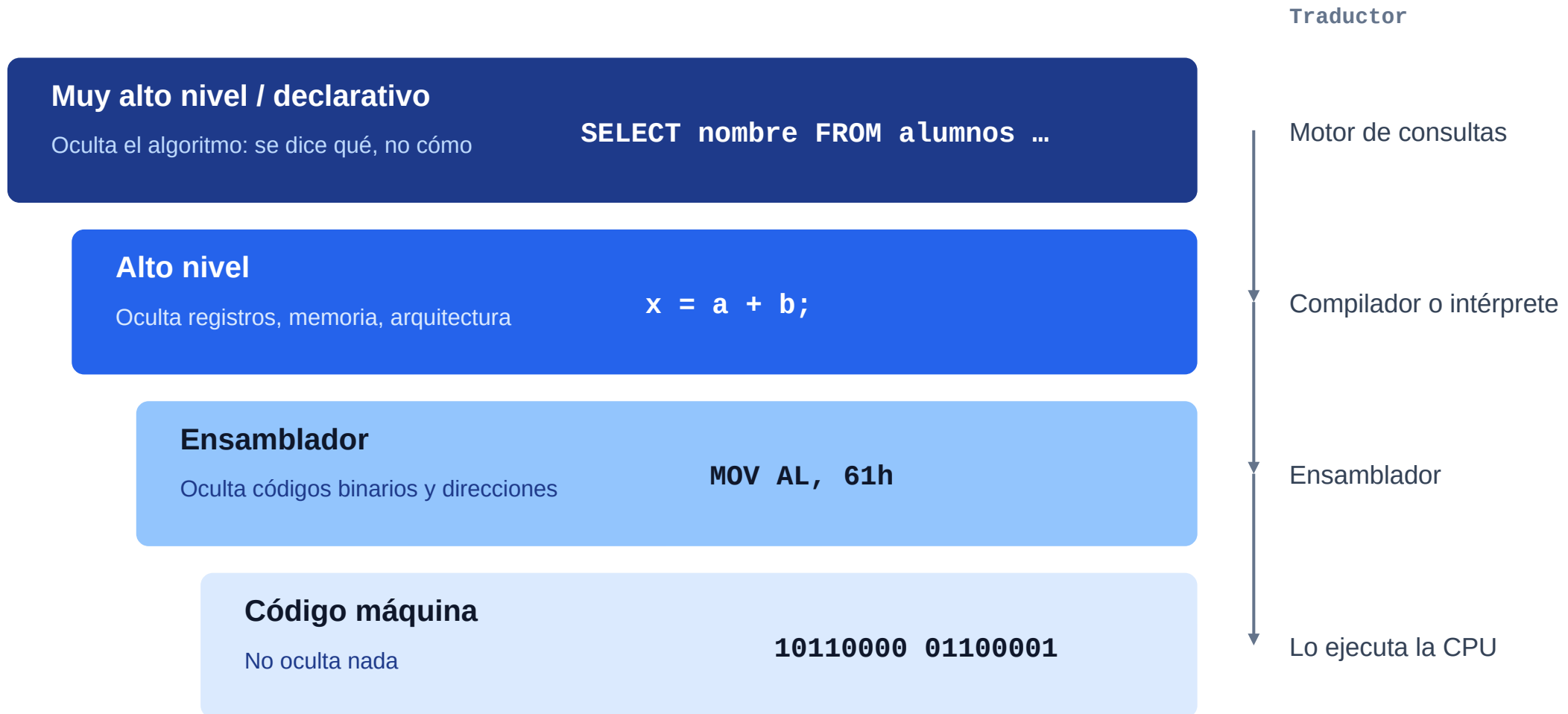
Abstracción de datos

Un tipo junto con sus operaciones, ocultando la representación.

TDA · clases · módulos

TDA: valores + operaciones, definidos por una especificación independiente de la implementación (Garrido, s.f., p. 6)

Niveles de abstracción



Subir un nivel = menos decisiones del programador, más decisiones del traductor.

El mismo cálculo en tres niveles

suma.asm · ensamblador x86-64

```
xor rax, rax
xor rcx, rcx
ciclo:
    cmp rcx, rdx
    jge fin
    add rax, [rsi+rcx*8]
    inc rcx
    jmp ciclo
fin:
```

9 líneas

suma.c · alto nivel

```
long suma(const long *xs,
          int n) {
    long total = 0;
    for (int i = 0; i < n; i++)
        total += xs[i];
    return total;
}
```

7 líneas

suma.py · muy alto nivel

```
total = sum(xs)
```

1 línea

✓ Menos errores

✓ Portabilidad

✓ Legibilidad

✓ Reutilización

Las abstracciones que usamos a diario

Variable

Abstrae la dirección de memoria

```
int edad = 20;
```

Tipo de dato

Valores + operaciones válidas

```
int · float · str
```

Control

Abstrae los saltos (Dijkstra, 1968)

```
if · while · for
```

Función

Se conoce la firma, no el cuerpo

```
def area(r): ...
```

Objeto

Estado + comportamiento

```
cuenta.depositar(100)
```

Clase

Plantilla de objetos

```
class Cuenta: ...
```

Módulo

Interfaz pública, implementación oculta (Parnas, 1972)

```
import math
```

Concurrencia

Tareas que se traslapan; CSP (Hoare, 1978)

```
async / await
```

Bajo nivel, alto nivel y dominio específico

Bajo nivel

Cerca de la máquina

EJEMPLOS

Código máquina, ensamblador

PORTABILIDAD

Nula

PRODUCTIVIDAD

Baja

USO

Firmware, embebidos, rendimiento crítico

Alto nivel

Cerca del programador

EJEMPLOS

C, Java, Python, Haskell

PORTABILIDAD

Alta

PRODUCTIVIDAD

Alta

EJECUCIÓN

Compilado · interpretado · híbrido
(bytecode + VM)

Dominio específico

Cerca del problema

EJEMPLOS

SQL, HTML, regex, LaTeX

PORTABILIDAD

Depende del motor

PRODUCTIVIDAD

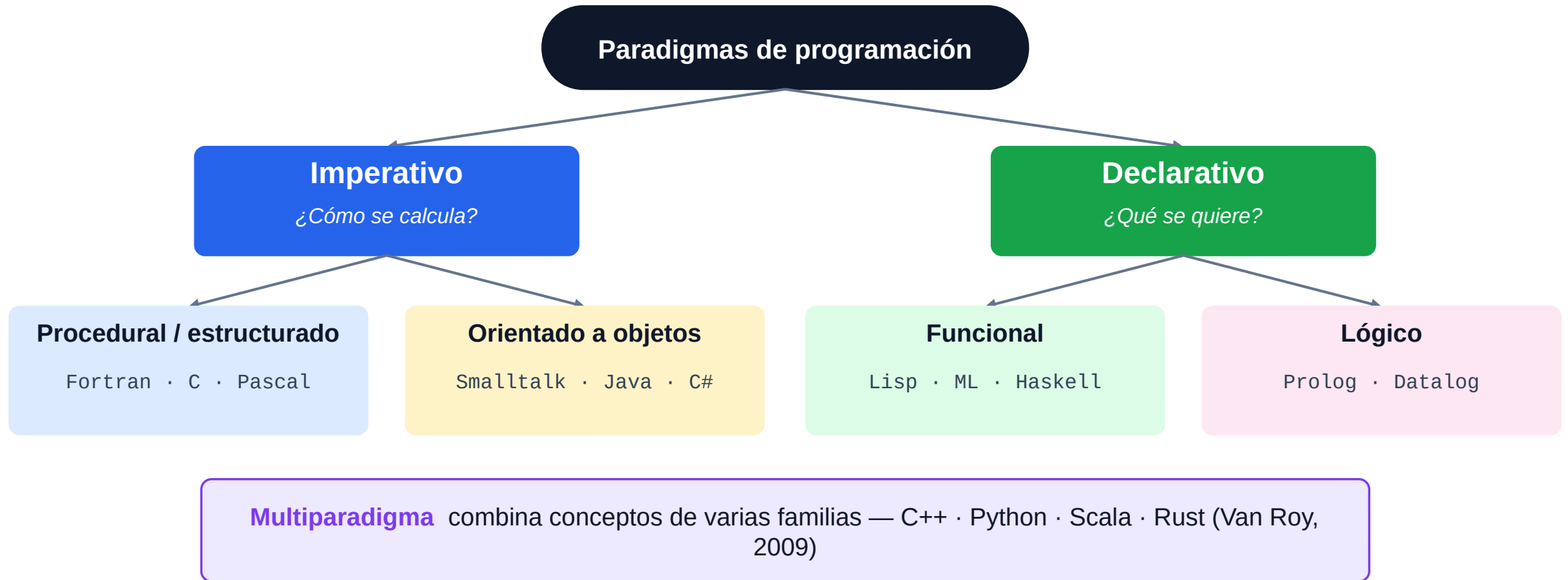
Muy alta en su dominio

VARIANTES

Externos (SQL) o internos (SQLAlchemy)

Fuentes: Sebesta (2019); Fowler (2010); Lenovo (s.f.); Zong et al. (2021)

Familias de paradigmas



Emergentes: reactivo · orientado a arreglos · probabilístico (Garg et al., 2026) · diferenciable

Cuatro formas de pensar un programa

Imperativo

¿Cómo se calcula?

UNIDAD

Instrucción

ESTADO

Mutable, explícito

FUNDAMENTO

Máquina de von Neumann

```
for (i = 2; i <= n; i++)  
  r *= i;
```

Backus (1978)

Objetos

¿Qué objetos colaboran?

UNIDAD

Objeto / clase

ESTADO

Mutable, encapsulado

FUNDAMENTO

Simulación, modelado

```
cuenta.depositar(100)
```

Kay (1993)

Funcional

¿Qué función transforma la entrada?

UNIDAD

Función pura

ESTADO

Inmutable

FUNDAMENTO

Cálculo lambda

```
factorial n =  
  n * factorial (n-1)
```

Hughes (1989)

Lógico

¿Qué es verdad?

UNIDAD

Hecho / regla

ESTADO

Sin estado explícito

FUNDAMENTO

Lógica de primer orden

```
abuelo(X,Z) :-  
  padre(X,Y), padre(Y,Z).
```

Colmerauer y Roussel (1993)

Lo que nos llevamos

1

La historia de los lenguajes es la historia de la abstracción

Del binario al ensamblador, al alto nivel y a lo declarativo: cada paso transfirió decisiones del programador al traductor.

2

Los paradigmas se complementan

Los lenguajes modernos son multiparadigma; el programador competente elige el concepto adecuado para cada parte (Van Roy, 2009).

3

Los lenguajes formales son la base técnica

Gramáticas libres de contexto para la sintaxis; expresiones regulares y autómatas para el análisis léxico.

Referencias (1/2)

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2.^a ed.). Addison-Wesley.

Al Mazrouei, S. K. S. (2025). Anka: A domain-specific language for reliable LLM code generation [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2512.23214>

Backus, J. (1978). Can programming be liberated from the von Neumann style? *Communications of the ACM*, 21(8), 613–641. <https://doi.org/10.1145/359576.359579>

Backus, J. W., et al. (1957). The FORTRAN automatic coding system. En *Papers presented at the Western Joint Computer Conference* (pp. 188–198). ACM. <https://doi.org/10.1145/1455567.1455599>

Colmerauer, A., & Roussel, P. (1993). The birth of Prolog. *ACM SIGPLAN Notices*, 28(3), 37–52. <https://doi.org/10.1145/155360.155362>

Dijkstra, E. W. (1968). Go To statement considered harmful. *Communications of the ACM*, 11(3), 147–148. <https://doi.org/10.1145/362929.362947>

Floyd, R. W. (1979). The paradigms of programming. *Communications of the ACM*, 22(8), 455–460. <https://doi.org/10.1145/359138.359140>

Fowler, M. (2010). *Domain-specific languages*. Addison-Wesley.

Garg, P., Geh, R. L., Israel, D., Millstein, T., Richardson, K., & Van den Broeck, G. (2026). Probabilistic programs of thought [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2604.17290>

Garrido, A. (s.f.). *Abstracción en programación* [Apuntes de curso]. DECSAI, Universidad de Granada. <https://ccia.ugr.es/~jfv/abstraccion.pdf>

Han, H. (2026). Mojo: A promising tool for scalable financial AI efficiency [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2606.16059>

Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, 21(8), 666–677. <https://doi.org/10.1145/359576.359585>

Referencias (2/2)

Huang, J., Li, Z., Li, Y., & Wang, H. (2025). NineToothed: A Triton-based high-level domain-specific language for machine learning [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2507.11978>

Hughes, J. (1989). Why functional programming matters. *The Computer Journal*, 32(2), 98–107. <https://doi.org/10.1093/comjnl/32.2.98>

Kay, A. C. (1993). The early history of Smalltalk. *ACM SIGPLAN Notices*, 28(3), 69–95. <https://doi.org/10.1145/155360.155364>

Lenovo. (s.f.). ¿Qué es un lenguaje de alto nivel? Glosario Lenovo. <https://www.lenovo.com/co/es/glosario/lenguaje-de-alto-nivel/>

Louden, K. C., & Lambert, K. A. (2011). *Programming languages: Principles and practice* (3.^a ed.). Cengage Learning.

McCarthy, J. (1960). Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4), 184–195. <https://doi.org/10.1145/367177.367199>

MSMK University College. (2025, 14 de febrero). Algol. Top Up Grado, MSMK. <https://msmk.university/algol/>

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>

Rugiero Jiménez, A. (2026). Vista general de los lenguajes de programación [Libreta de Gemini Notebook]. Google.

Sebesta, R. W. (2019). *Concepts of programming languages* (12.^a ed.). Pearson.

Van Roy, P. (2009). Programming paradigms for dummies: What every programmer should know. En G. Assayag & A. Gerzso (Eds.), *New computational paradigms for computer music* (pp. 9–47). IRCAM/Delatour France.

Zong, J., Pollock, J., Wootton, D., & Satyanarayan, A. (2021). Design spaces of domain-specific languages: Comparing and contrasting approaches in PL and HCI [Ponencia]. PLATEAU Workshop. <https://vis.csail.mit.edu/pubs/dsl-design-spaces/>

¿Preguntas?

```
total = sum(xs)    # la abstracción, en una línea
```

Angel Rugerio Jiménez — 201720 · Lenguajes de Computación · Otoño 2026